OPTIMIZER WARS: SELECTION

A Scientific Investigation into Gradient Descent Optimizers | ID:24178436

***GitHub Link**: https://github.com/MoazTahir/Optimizer-Wars*

When you are working on a DL based project, you start writing PyTorch code, then type "optimizer="", and pause. SGD? Adam? AdamW? Which one to go for? At first this choice feels arbitrary and you should just choose the mainstream most accurate one in terms of accuracy, until you watch one model converge in 8 epochs while the other one is still under 80% accuracy at epoch 20. This tutorial is a means to make that difference visible and teaches the key differences between different optimizers and where to use which one.

1. Introduction

An **optimizer** is the core component that decides how a neural network's weights are updated after each backward pass. The update rule encodes assumptions about the loss landscape and sees few things: noise level of gradients, whether curvature differs across directions, and current gradient versus history comparison. Each optimizer in this tutorial aims to solve a specific failure mode of its predecessor. This helps understanding how each function, rather than memorizing hyperparameters. We ran an experiment on **FashionMNIST** using a same CNN, same weight initialization, and same learning rate for all five optimizers. The only variable allowed to change was the update rule. This tutorial aims on further enhancing your knowledge on optimizers on a deeper level.

2. What is an Optimizer?

So, when training, we aim on finding weights **θ** that minimize a loss L(θ). We can visualize this loss landscape as a higher dimensional surface where each axis is one weight value and the vertical axis represents the loss values. The optimizer aims on going downhill (or decreasing), trying to minimize the loss. However real landscapes aren't smooth bowls as they contain **ravines**, **saddle points**, and wide flat regions. Each optimizer referred to below was designed to handle one or more of these geometries or scenarios.

3. The Optimizers: Theory

***Running analogy:** Consider a hiker (traveler across uneven terrain) is descending a mountain in dense fog, and he is navigating only by the slope felt underfoot meaning what limited knowledge he has of his immediate surrounding. The mountain is the loss landscape (plateaus and valleys), where the hiker's current position is the model weights, and the valley floor is the minimum where the loss is the least. Each optimizer is a different movement strategy for the hiker. The terrain however never changes, only the decision rule does.*

3.1 Stochastic Gradient Descent: Step Purely on the Current Slope

At each step, we measure the slope underfoot across all directions and take a fixed step downhill from where we stand. There is no memory nor any adaptation involved. The gradient gives direction to go and the learning rate η gives step size (the number of steps for analogy sakes).

The ravine problem. Imagine the hiker on a narrow ridge running north toward the valley, but that ridge is accompanied with steep east-west cliffs. The gradient points mostly east-west (that's the steep face) rather than north (the productive direction where the actual true minima lies). The hiker decides to zigzag violently across the ridge, while barely advancing.

This is exactly what SGD does in the ravines that are common in deep network loss landscapes. Gradients across the narrow dimension dominate in such cases, and most of each step is redundantly wasted on oscillation. Additionally, the learning rate compounds this, if its too large the hiker may end up bouncing off the cliff. If too small, 20 epochs barely move them. Despite this shortcoming, SGD with a cosine schedule remains a competitive ceiling for final accuracy because it's simple steps tend to land in wider, flatter valleys that generalize better unlike standard SGD (He et al., 2016).

3.2 SGD + Momentum: Carry Inertia from Past Steps

Now give the hiker a heavy rucksack. For each step, the rucksack's inertia carries them slightly in the direction they were already moving. In this case Momentum (Polyak, 1964) accumulates a **velocity vector** v, a decaying sum of past gradients. The update rides this velocity, not the raw gradient like SGD.

Back on the ridge: with $\mu = 0.9$, the east-west oscillations alternate direction every step and cancel in the velocity due to the rucksack's inertia, while the consistent northward movement accumulates. The hiker, in this case, easily accelerates along the ridge and the zigzag collapses into a smooth curve. However, the flipside of the rucksack is **overshoot** on a long slope. The hiker builds enough inertia to barrel past the valley floor before the reversed gradient can slow them down. In actual training, this appears as oscillation near the minimum late in optimization. Momentum also adds μ as a second hyperparameter to tune, though 0.9 is robust across most problems according to literature.

3.3 RMSProp: Adapt Stride Length Per Direction

Previously momentum fixed the direction problem but not a deeper one. What if some dimensions are inherently much steeper/deeper than others? A fixed η usually overshoots steep weights and stalls flat ones. Adagrad (Duchi et al., 2011) tried to fix this by dividing each update by the sum of all past squared gradients, however even with that the denominator only grew, until learning stops. RMSProp (Hinton, 2012) then further replaces that sum with a **running average**, so old gradients fade.

Now, the hiker now has a different stride length per compass direction. The steep north-south face has seen large recent gradients, the value of $\mathrm{sqrt}(E[g^2])$ becomes large, so now the effective step would be small and cautious as to not go over. On the flipside, the gentle east-west floor has small gradients, $\mathrm{sqrt}(E[g^2])$ is small, so the hiker strides confidently and can take bigger steps. With $\rho = 0.99$, the hiker adapts to new terrain rather than staying permanently slowed by a steep patch crossed hours ago and being overly cautious. The remaining weakness is that RMSProp tracks gradient *magnitude* but not *direction*. There is no concept of inertia to smooth the path, so near convergence is visible. When the gradients become small and noisy, the trajectory spirals erratically.

3.4 Adam: Inertia and Adaptive Stride, Combined

So, Momentum solves direction, RMSProp solves scaling and being cautious. Adam (Kingma & Ba, 2015) does both. It tracks a **first moment** m (decaying gradient average, like momentum) and a **second moment** v (decaying squared-gradient average, like RMSProp), then divides one by the other so its best of both worlds. It also adds a **bias correction**. Which refers to both moments to be initialized at zero. This aids in early estimates that are severely

biased downward. Without correction, the first few steps use near-zero moment estimates, then suddenly overshoot as the estimates warm up.

Back on our ridge, Adam's hiker carries the rucksack (smooth direction) *and* adapts stride per terrain (no overshoot on steep faces) and calibrates both from the very first step. This combined approach results in a fast, stable convergence across a wide range of learning rates. Our experiments show Adam tolerates a 100× LR range where SGD collapses outside a 5× window. With all that's good, theres a hidden flaw, weight decay (L2 regularization) is supposed to shrink all weights uniformly. But in Adam+L2, the decay term is added to the gradient *before* the adaptive divide, so it too gets divided. Resulting in non uniform shrinking of weights. This simply means historically larger/ higher variance gradients receive least decay and vice versa.

3.5 AdamW: Decouple the Weight Decay

AdamW (Loshchilov & Hutter, 2019) fixes Adam's regularization with one change: Which is to apply weight decay **after** the adaptive gradient step, not inside it. Which is exactly the problem mentioned at the end of previous optimizer.

In Adam+L2, the guide tells the hiker: 'trim your pack but scale the trimming to how small your stride is in each direction'. The fast-moving directions (large v*hat, short adaptive stride) get almost no trimming; the slow ones (small v*hat, long stride) get over-trimmed. This is what L2 regularization was always supposed to mean. AdamW is now the default for BERT, GPT, ViT, and every large model. Our overfitting experiment (Section 5.4) reproduces this directly.

4. Experiment Setup

Dataset: FashionMNIST is chosen over classic MNIST since it is complex enough that optimizer choice is visible, but is fast enough to run all five optimizers in minutes.

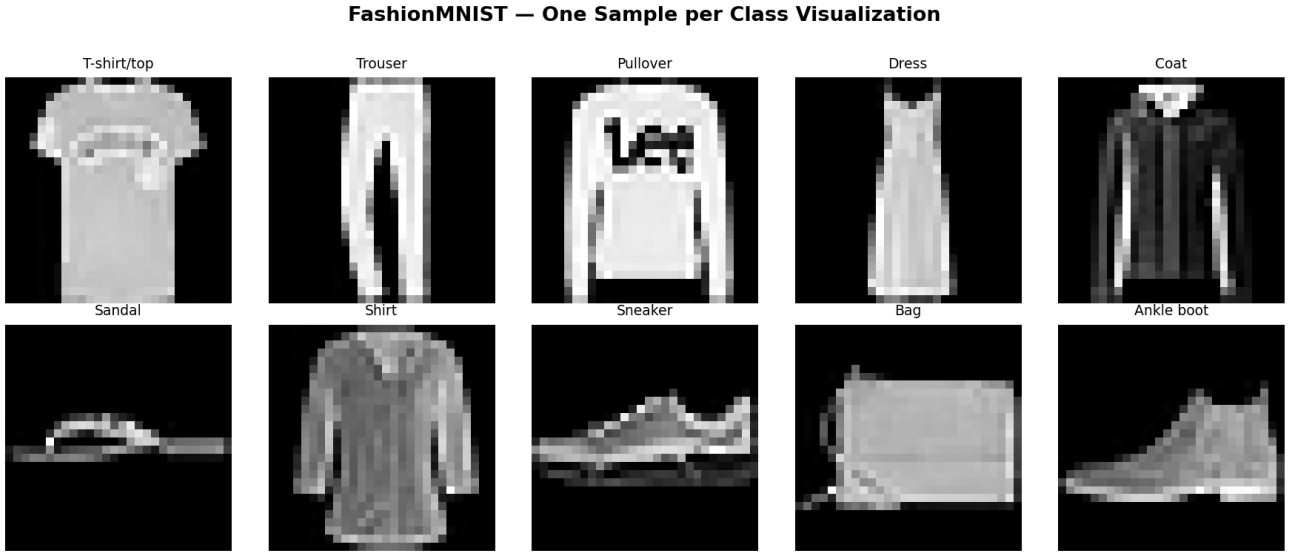

Figure 0. A Grid of ten grayscale 28x28 FashionMNIST images as example, one per class.

Model: It consists of two convolutional blocks which feed to a linear classifier with dropout. 128 batch size, 10 epochs, learning rate 0.001 are the remaining common hyper parameters. AdamW has weight decay (λ=10^-4) for regularization effects.

5. Results and Discussion

5.1 Convergence Speed

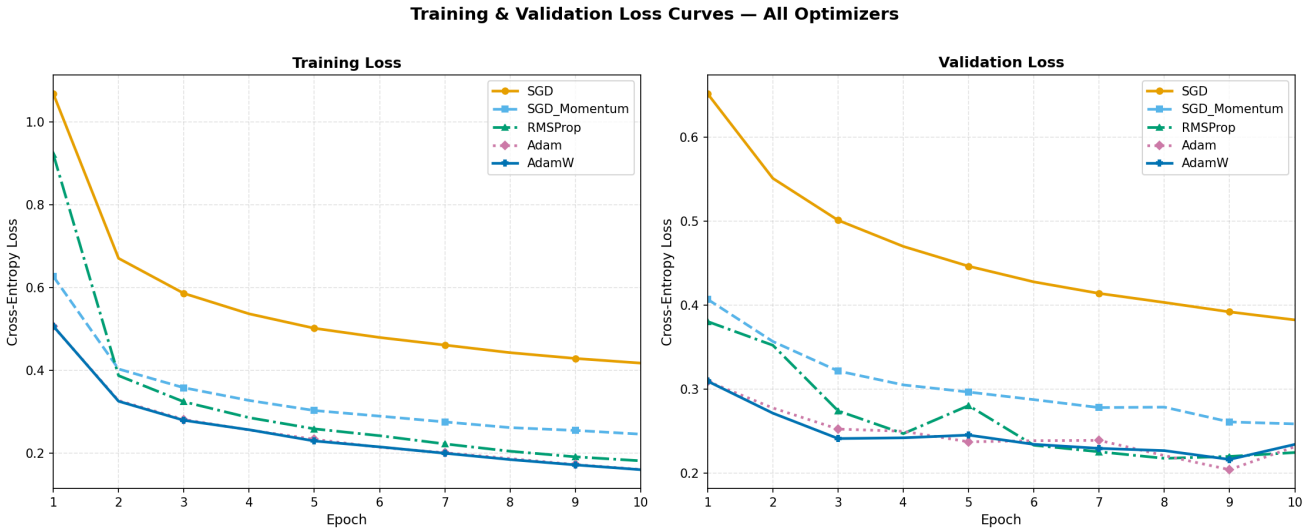

Figure 1. Two-line charts above show training loss (left) and validation loss (right) over 10 epochs for five optimizers. Here, Adaptive methods (Adam, AdamW, RMSProp) descend steeply in the first three epochs while SGD remains highest throughout.

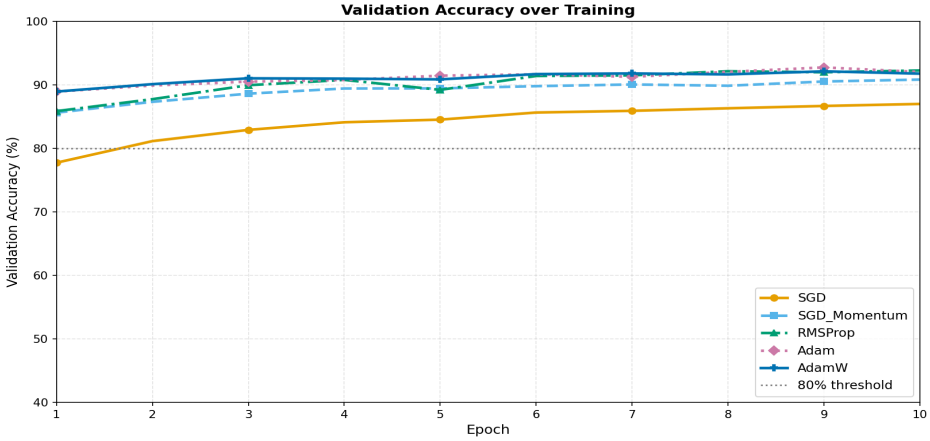

Figure 2.1. Above shows Line chart of validation accuracy over 10 epochs for all five optimizers. Adam and AdamW once again outperform the rest initially.

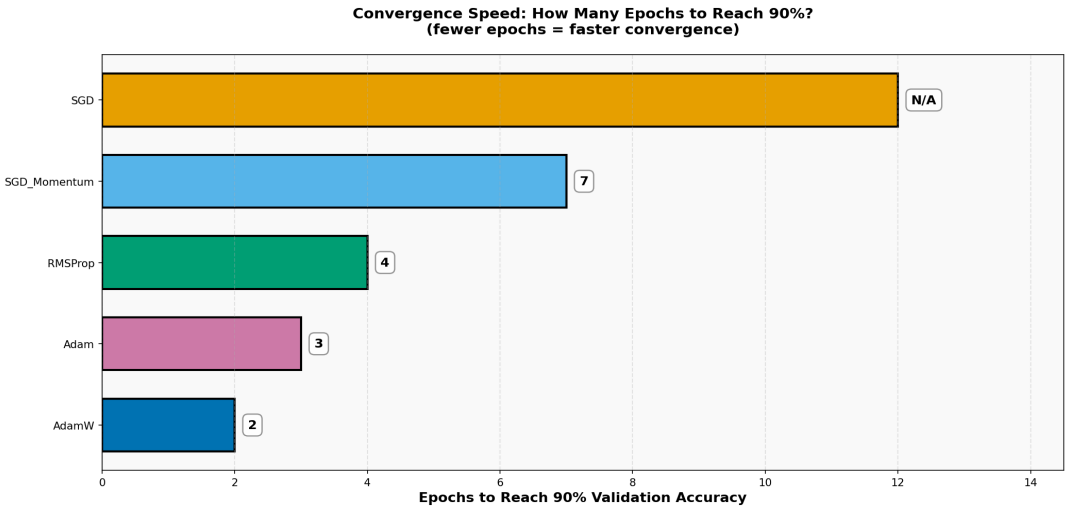

Figure 2.2. The above Horizontal bar chart shows epochs required to reach optimal threshold accuracy (90+) for each optimizer. The lesser the num of epochs, the faster the convergence.

5.2 Learning Rate Sensitivity

From the heatmap, we can deduce that Vanilla SGD is the most brittle as its comfortable range spans less than one order of magnitude (too high = diverges, too low = stalls). Adam and AdamW perform well across the full 100× tested range. This robustness is why adaptive methods dominate in practice, they have fewer tuning runs, lower compute cost, and faster iteration.

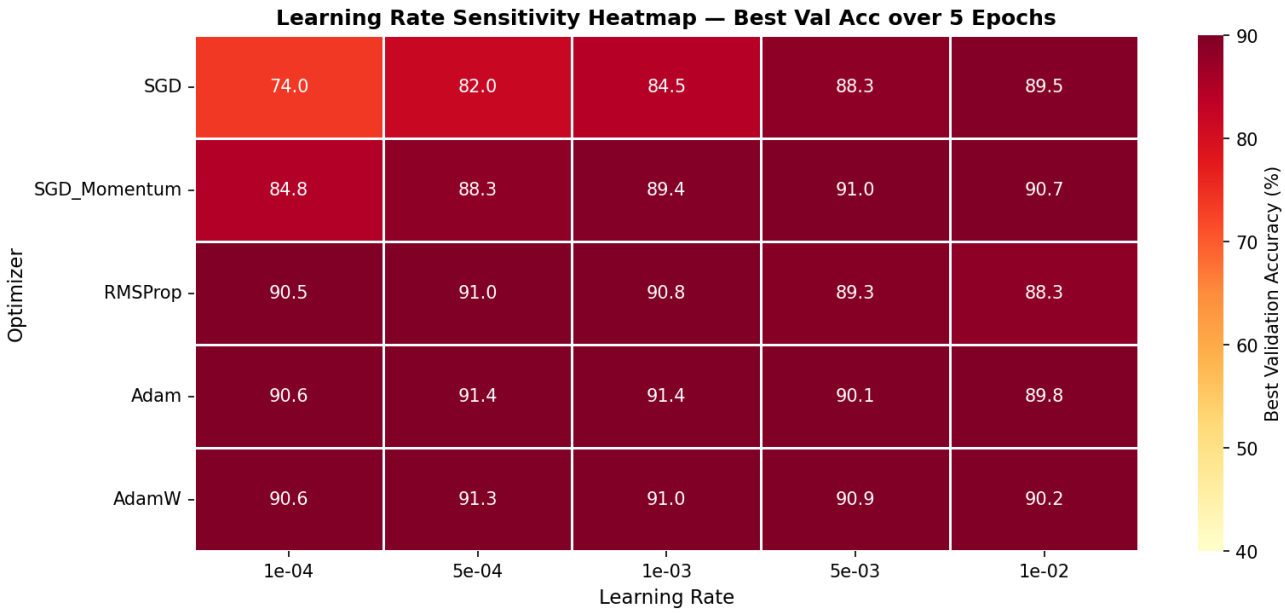

Figure 3. The above Heatmap shows optimizers on the vertical axis and learning rates from 1e-4 to 1e-2 on the horizontal axis over 5 epochs of training data. The Cell values are best validation accuracy percentages for those instances. As before, Adam and AdamW show high accuracy across the full learning rate range while SGD accuracy collapses at higher learning rates.

5.3 Loss Landscape Trajectories

So now 3-layer MLP was trained on a spiral dataset, and the parameter snapshots were recorded throughout. Along that, PCA-projected trajectories were shared to a 2D subspace. The geometric motions are described in alt texts below in 4.2.

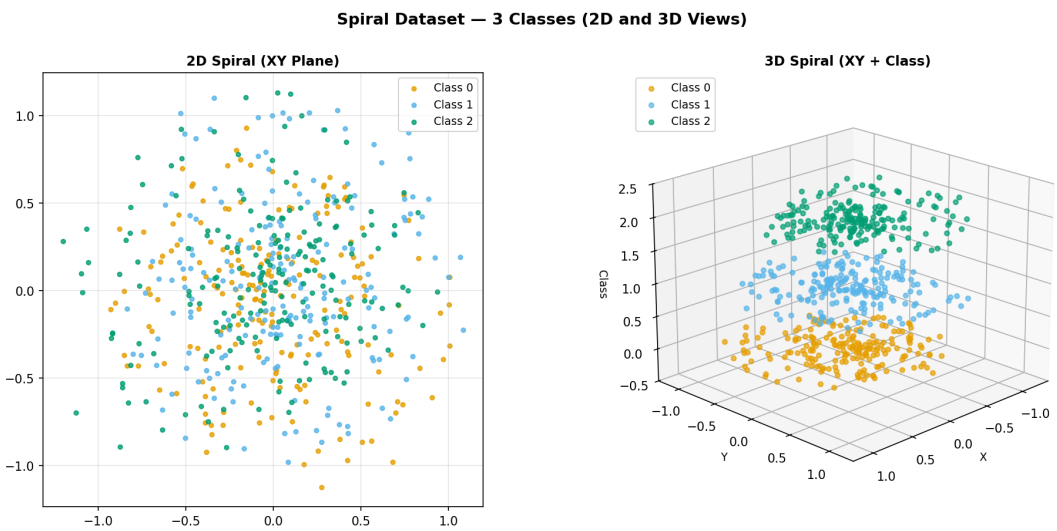

Figure 4.1. Above two are Scatter plot of a three-class spiral dataset in 2D and 3D for visualization and understanding

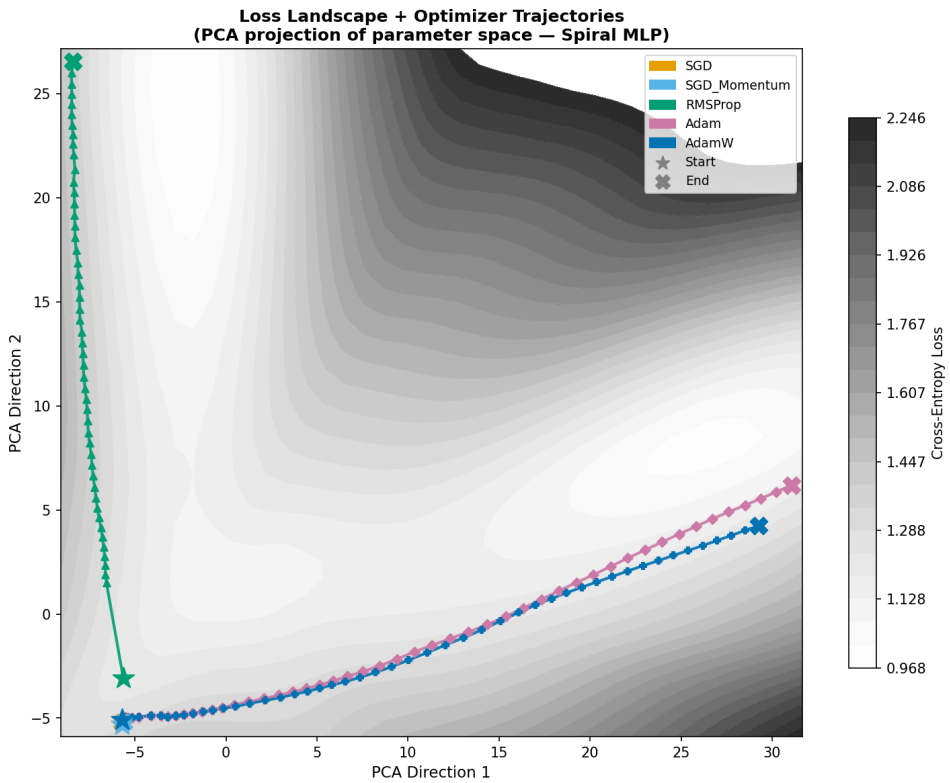

Figure 4.2. A Contour plot of a 2D PCA-projected loss surface overlaid with optimizer training trajectories is visualized above. The stars are the start point and crosses are the endpoints (where convergence occurred).

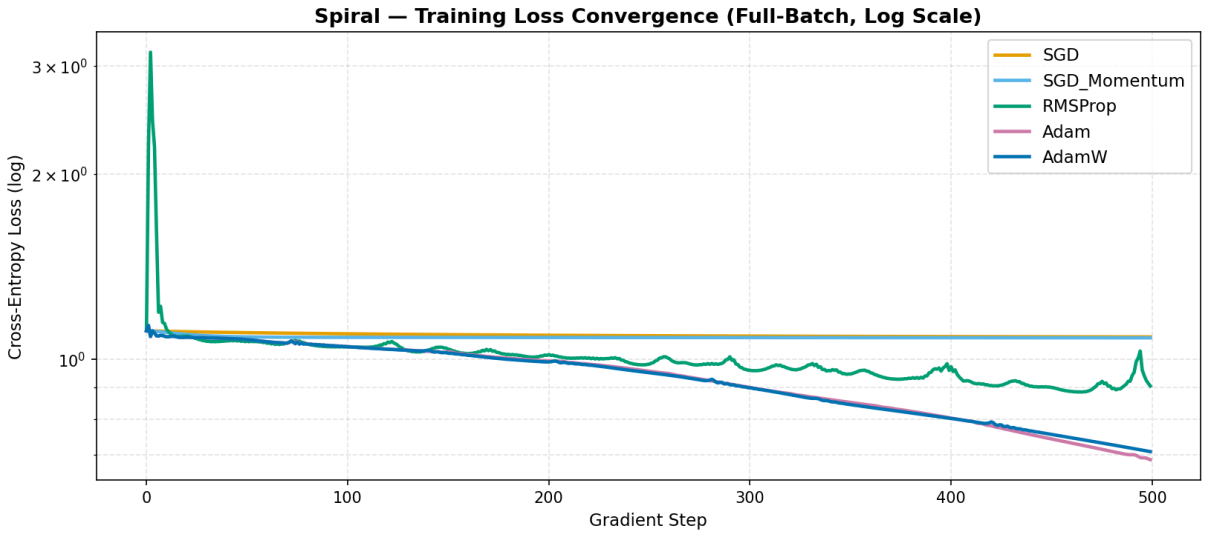

Figure 4.3 The above Line chart of training loss is spanned over 500 gradient steps for the spiral MLP

5.4 Adam vs AdamW: Overfitting Experiment

To isolate the regularization difference, the model is retrained with 20% data (no dropout layer) and weight decay quality becomes the deciding factor and we can compare the both approaches effectively.

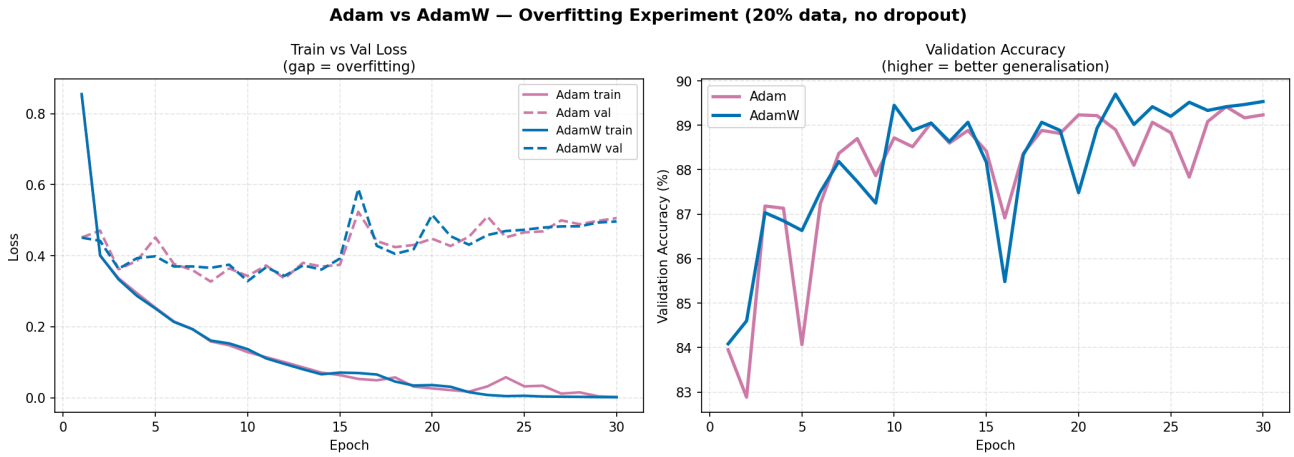

Figure 5 Left grap shows Adam's growing Train-Val gap while Right showcases validation accuracy over epochs. It is visible that AdamW maintains a higher and more stable accuracy than standard Adam.

6. Practical Decision Framework with Findings

The table below summarizes when to reach for each optimizer, based on the experiments and literature:

Optimizer	Best for	LR to try first	Key risk
SGD	Debugging, baselines	0.01 – 0.1	Very LR-sensitive
SGD+Momentum	Final accuracy, long schedules	0.01 + cosine	Overshoots; needs tuning
RMSProp	RNNs, RL, time series	1e-4 – 1e-3	Unstable late training
Adam	Rapid prototyping	1e-3	Overfits; broken L2
AdamW	Everything else (default)	1e-4 – 3e-4	Slightly more to tune

One-line rule: Make sure to start off with AdamW as your starting point in prototyping. Then on Later stages if you have the compute budget, go for Switch to SGD+Momentum+ cosine schedule. You may use RMSProp for recurrent networks and reinforcement learning as it stabilizes gradients over time, helping avoid exploding/vanishing gradients (RNN) and handles high variance (RL).

7. Conclusions

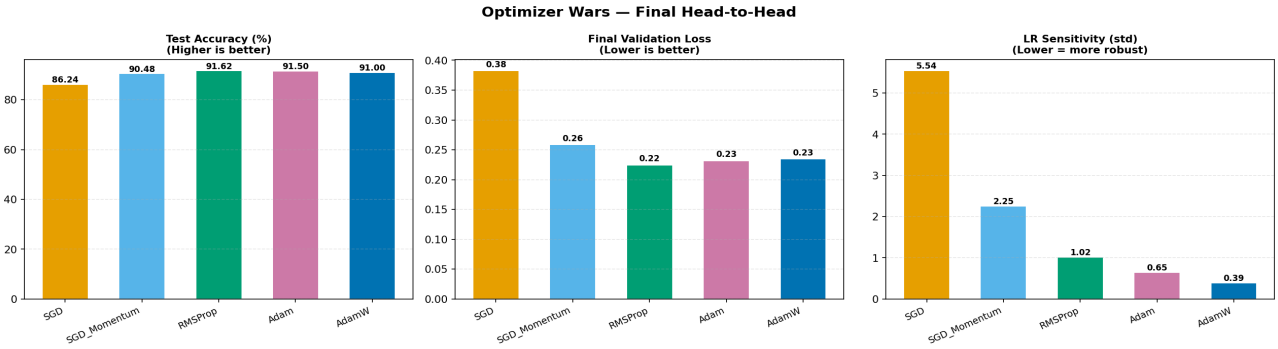

Figure 6. Three bar charts compare all five optimizers on test accuracy, final validation loss, and learning rate sensitivity. AdamW and Adam both score highest, and the SGD shows the highest sensitivity variance.

Five experiments produced five clear findings:

- Adaptive methods converge 2–5× faster than SGD variants under identical conditions.
- Speed does not correlate to generalization as Adam converges fastest but shows the largest train-validation gap.
- SGD+Momentum is not obsolete as it is slower to converge, but with a proper schedule and much needed compute, it often achieves the best final accuracy and is the standard result in competitive image classification benchmarks.
- LR sensitivity is the biggest practical risk as Vanilla SGD collapses with a 5× LR error whilst the Adam optimizers tolerate a 100× range making Adam dominate in practice.
- Trajectory visualization shows momentum methods traverse the landscape smoothly, vanilla SGD zigzags, RMSProp spirals tightly. All in all, geometry does in fact predict performance.

In short, **AdamW is the right default** for most deep learning tasks. Not because it is always the best, but because it is rarely far from the best and requires the least tuning to get there.

References

Duchi, J., Hazan, E. and Singer, Y. (2011) ‘Adaptive Subgradient Methods for Online Learning and Stochastic Optimization’, Journal of Machine Learning Research, 12, pp. 2121–2159. Available at: https://jmlr.org/papers/v12/duchi11a.html

He, K., Zhang, X., Ren, S. and Sun, J. (2016) ‘Deep Residual Learning for Image Recognition’, in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, June 2016, pp. 770–778. Available at: https://arxiv.org/abs/1512.03385

Kingma, D.P. and Ba, J. (2015) ‘Adam: A Method for Stochastic Optimization’, in 3rd International Conference on Learning Representations (ICLR 2015), San Diego. Available at: https://arxiv.org/abs/1412.6980

Li, H., Xu, Z., Taylor, G., Studer, C. and Goldstein, T. (2018) ‘Visualizing the Loss Landscape of Neural Nets’, in Advances in Neural Information Processing Systems 31 (NeurIPS 2018). Available at: https://arxiv.org/abs/1712.09913

Loshchilov, I. and Hutter, F. (2019) ‘Decoupled Weight Decay Regularization’, in 7th International Conference on Learning Representations (ICLR 2019), New Orleans. Available at: https://arxiv.org/abs/1711.05101

Polyak, B.T. (1964) ‘Some methods of speeding up the convergence of iteration methods’, USSR Computational Mathematics and Mathematical Physics, 4(5), pp. 1–17. doi: 10.1016/0041-5553(64)90137-5

Ruder, S. (2016) ‘An Overview of Gradient Descent Optimization Algorithms’, arXiv preprint arXiv:1609.04747 [cs.LG]. Available at: https://arxiv.org/abs/1609.04747

Sutskever, I., Martens, J., Dahl, G. and Hinton, G. (2013) ‘On the Importance of Initialization and Momentum in Deep Learning’, in Proceedings of the 30th International Conference on Machine Learning (ICML 2013), Proceedings of Machine Learning Research, 28(3), pp. 1139–1147. Available at: https://proceedings.mlr.press/v28/sutskever13.html

Tieleman, T. and Hinton, G. (2012) ‘Lecture 6.5 - RMSProp: Divide the Gradient by a Running Average of Its Recent Magnitude’, COURSERA: Neural Networks for Machine Learning, Lecture Notes. University of Toronto.

Xiao, H., Rasul, K. and Vollgraf, R. (2017) ‘Fashion-MNIST: A Novel Image Dataset for Benchmarking Machine Learning Algorithms’, arXiv preprint arXiv:1708.07747 [cs.LG]. Available at: https://arxiv.org/abs/1708.07747